

```
-----  
  
-- Lookup Tables  
  
-----
```

```
CREATE TABLE roles (  
  
    role_id SERIAL PRIMARY KEY,  
  
    role_name VARCHAR(50) UNIQUE NOT NULL  
  
);
```

```
CREATE TABLE genders (  
  
    gender_id SERIAL PRIMARY KEY,  
  
    gender_name VARCHAR(20) UNIQUE NOT NULL  
  
);
```

```
CREATE TABLE appointment_status (  
  
    status_id SERIAL PRIMARY KEY,  
  
    status_name VARCHAR(20) UNIQUE NOT NULL  
  
);
```

```
CREATE TABLE reminder_types (  
  
    reminder_type_id SERIAL PRIMARY KEY,  
  
    type_name VARCHAR(20) UNIQUE NOT NULL  
  
);
```

```
CREATE TABLE reminder_status (  
  
    reminder_status_id SERIAL PRIMARY KEY,  
  
    status_name VARCHAR(20) UNIQUE NOT NULL  
  
);
```

```
CREATE TABLE payment_methods (  
  
    method_id SERIAL PRIMARY KEY,  
  
    method_name VARCHAR(50) UNIQUE NOT NULL  
  
);
```

```
CREATE TABLE payment_status (  
  
    status_id SERIAL PRIMARY KEY,  
  
    status_name VARCHAR(20) UNIQUE NOT NULL  
  
);
```

```
CREATE TABLE blood_group (  
  
    blood_group_id SERIAL PRIMARY KEY,  
  
    blood_group_name VARCHAR(20) UNIQUE NOT NULL  
  
);
```

```
CREATE TABLE divisions (  
  
    division_id SERIAL PRIMARY KEY,  
  
    division_name VARCHAR(100) UNIQUE NOT NULL  
  
);
```

```
CREATE TABLE districts (  
  
    district_id SERIAL PRIMARY KEY,  
  
    district_name VARCHAR(100) NOT NULL,  
  
    division_id INT NOT NULL REFERENCES divisions(division_id) ON DELETE  
CASCADE,  
  
    UNIQUE(district_name, division_id)  
  
);
```

```
CREATE TABLE upazilas (  

```

```
    upazila_id SERIAL PRIMARY KEY,  
  
    upazila_name VARCHAR(100) NOT NULL,  
  
    district_id INT NOT NULL REFERENCES districts(district_id) ON DELETE  
CASCADE,  
  
    UNIQUE(upazila_name, district_id)  
);
```

```
CREATE TABLE symptoms (  
  
    symptom_id SERIAL PRIMARY KEY,  
  
    symptom_name VARCHAR(100) UNIQUE NOT NULL,  
  
    symptom_image VARCHAR(255)  
);
```

```
CREATE TABLE departments (  
  
    department_id SERIAL PRIMARY KEY,  
  
    department_name VARCHAR(100) UNIQUE NOT NULL,  
  
    department_image VARCHAR(255)  
);
```

```
-- Core User Tables
```

```
-- -----
```

```
CREATE TABLE users (  
  
    user_id SERIAL PRIMARY KEY,  
  
    first_name VARCHAR(50),  
  
    last_name VARCHAR(50),  
  
    email VARCHAR(100) UNIQUE NOT NULL,  
  
    password_hash VARCHAR(255) NOT NULL,  
  
    phone VARCHAR(20),  
  
    role_id INT NOT NULL REFERENCES roles(role_id),  
  
    created_at TIMESTAMP DEFAULT NOW(),  
  
    updated_at TIMESTAMP DEFAULT NOW()  
);
```

```
CREATE TABLE patients (  
  
    patient_id INT PRIMARY KEY REFERENCES users(user_id) ON DELETE  
CASCADE,  
  
    date_of_birth DATE,  
  
    gender_id INT REFERENCES genders(gender_id),
```

```
division_id INT REFERENCES divisions (division_id),

district_id INT REFERENCES districts (district_id),

upazila_id INT REFERENCES upazilas (upazila_id),

exact_location VARCHAR(255),

blood_group_id INT REFERENCES blood_group (blood_group_id),

medical_history TEXT

);

CREATE TABLE doctors (

    doctor_id INT PRIMARY KEY REFERENCES users (user_id) ON DELETE CASCADE,

    about TEXT,

    bmdc_number VARCHAR(255),

    qualification VARCHAR(255),

    total_experience INT DEFAULT 0,

    working_hours JSONB,

    profile_image VARCHAR(255),

    is_verified BOOLEAN DEFAULT FALSE,

    division_id INT REFERENCES divisions (division_id),

    district_id INT REFERENCES districts (district_id),
```

```

        upazila_id INT REFERENCES upazilas(upazila_id)

);

CREATE TABLE doctor_experience (

    id SERIAL PRIMARY KEY,

    doctor_id INT NOT NULL REFERENCES doctors(doctor_id) ON DELETE
CASCADE,

    hospital_name VARCHAR(255),

    designation VARCHAR(255),

    department VARCHAR(255)

);

CREATE TABLE doctor_specialization_departments (

    doctor_id INT NOT NULL REFERENCES doctors(doctor_id) ON DELETE
CASCADE,

    department_id INT NOT NULL REFERENCES departments(department_id) ON
DELETE CASCADE,

    PRIMARY KEY (doctor_id, department_id)

);

CREATE TABLE doctor_specialization_symptoms (

```

```

        doctor_id INT NOT NULL REFERENCES doctors(doctor_id) ON DELETE
CASCADE,

        symptom_id INT NOT NULL REFERENCES symptoms(symptom_id) ON DELETE
CASCADE,

        PRIMARY KEY (doctor_id, symptom_id)
);

CREATE TABLE admins (

        admin_id INT PRIMARY KEY REFERENCES users(user_id) ON DELETE CASCADE,

        role_description TEXT
);

-- -----
-- Appointment & Consultation
-- -----

CREATE TABLE appointments (

        appointment_id SERIAL PRIMARY KEY,

        patient_id INT NOT NULL REFERENCES patients(patient_id),

        doctor_id INT NOT NULL REFERENCES doctors(doctor_id),

```



```

        appointment_date DATE NOT NULL,

        appointment_time TIME NOT NULL,

        status_id INT REFERENCES appointment_status(status_id),

        reason TEXT,

        created_at TIMESTAMP DEFAULT NOW(),

        updated_at TIMESTAMP DEFAULT NOW(),

        UNIQUE(patient_id, appointment_date, appointment_time),

        UNIQUE(doctor_id, appointment_date, appointment_time)

);

CREATE TABLE reminders (

    reminder_id SERIAL PRIMARY KEY,

    appointment_id INT NOT NULL REFERENCES appointments(appointment_id) ON
DELETE CASCADE,

    user_id INT NOT NULL REFERENCES users(user_id),

    reminder_type_id INT NOT NULL REFERENCES
reminder_types(reminder_type_id),

    message TEXT NOT NULL,

    sent_at TIMESTAMP,

    reminder_status_id INT REFERENCES reminder_status(reminder_status_id)
DEFAULT 1

```

```
);

CREATE TABLE prescriptions (

    prescription_id SERIAL PRIMARY KEY,

    appointment_id INT NOT NULL REFERENCES appointments(appointment_id) ON
DELETE CASCADE,

    doctor_id INT NOT NULL REFERENCES doctors(doctor_id),

    patient_id INT NOT NULL REFERENCES patients(patient_id),

    prescription_text TEXT NOT NULL,

    issued_at TIMESTAMP DEFAULT NOW()

);

CREATE TABLE doctor_reviews (

    review_id SERIAL PRIMARY KEY,

    doctor_id INT NOT NULL REFERENCES doctors(doctor_id),

    patient_id INT NOT NULL REFERENCES patients(patient_id),

    rating INT CHECK (rating BETWEEN 1 AND 5),

    review_text TEXT,

    created_at TIMESTAMP DEFAULT NOW()

);
```

```
-- -----
```

```
-- Payments
```

```
-- -----
```

```
CREATE TABLE payments (  
  
    payment_id SERIAL PRIMARY KEY,  
  
    appointment_id INT REFERENCES appointments(appointment_id) ON DELETE  
CASCADE,  
  
    patient_id INT REFERENCES patients(patient_id),  
  
    doctor_id INT REFERENCES doctors(doctor_id),  
  
    amount NUMERIC(10,2) NOT NULL,  
  
    method_id INT REFERENCES payment_methods(method_id),  
  
    status_id INT REFERENCES payment_status(status_id),  
  
    transaction_time TIMESTAMP DEFAULT NOW()  
);
```

```
CREATE TABLE commissions (  
  
    commission_id SERIAL PRIMARY KEY,  
  
    payment_id INT REFERENCES payments(payment_id) ON DELETE CASCADE,
```

```

        admin_id INT REFERENCES admins(admin_id),

        commission_amount NUMERIC(10,2) NOT NULL,

        created_at TIMESTAMP DEFAULT NOW()

);

-----

-- Chat & Security

-----

CREATE TABLE chat_messages (

    message_id SERIAL PRIMARY KEY,

    appointment_id INT REFERENCES appointments(appointment_id) ON DELETE
CASCADE,

    sender_id INT REFERENCES users(user_id),

    receiver_id INT REFERENCES users(user_id),

    message_text TEXT,

    sent_at TIMESTAMP DEFAULT NOW()

);

CREATE TABLE call_sessions (

```

```
call_id SERIAL PRIMARY KEY,

appointment_id INT REFERENCES appointments(appointment_id),

doctor_id INT REFERENCES doctors(doctor_id),

patient_id INT REFERENCES patients(patient_id),

call_type VARCHAR(20) CHECK (call_type IN ('audio', 'video')),

started_at TIMESTAMP,

ended_at TIMESTAMP,

duration INTERVAL

);

CREATE TABLE audit_logs (

    log_id SERIAL PRIMARY KEY,

    user_id INT REFERENCES users(user_id),

    action VARCHAR(100),

    table_name VARCHAR(50),

    record_id INT,

    timestamp TIMESTAMP DEFAULT NOW(),

    details JSONB

);
```



```
$env:DJANGO_SETTINGS_MODULE = "medisched.settings"
```

```
***** setx DJANGO_SETTINGS_MODULE "medisched_app.settings"  
cmd /c "setx DJANGO_SETTINGS_MODULE ""  
cmd /c "setx DJANGO_SETTINGS_MODULE ""
```

1 User Management & Roles

Tables: `users`, `roles`, `admins`, `patients`, `doctors`

Feature:

- বিভিন্ন ধরনের user: Patient, Doctor, Admin
- User authentication & authorization (JWT / Session)
- Admins role-wise permission control
- Doctor / Patient profile management
- Profile info (name, email, phone, location)

Professional Realism:  High

- এই structure দিয়ে hospital/clinic system level user management করা যাবে।
-

2 Appointment Scheduling

Tables: `appointments`, `appointment_status`

Feature:

- Patient doctor এর সাথে appointment book করতে পারবে
- Doctor availability & schedule check
- Appointment status (confirmed, pending, canceled, completed)
- Date & time validation (এক doctor একই সময়ে দুই appointment নিতে পারবে না)

Professional Realism:  High

- Critical for real-life telemedicine apps
-

3 Reminders / Notifications

Tables: `reminders`, `reminder_types`, `reminder_status`

Feature:

- Appointment এর আগে patient/doctor কে reminder পাঠানো (email/SMS/push)
- Reminder tracking (sent, pending, failed)

Professional Realism:  Medium-High

- Requires background tasks (Celery + Redis)
- Real-life apps যেমন Practo, ZocDoc এ এই feature আছে

4 Doctor Profile & Specializations

Tables: `doctors`, `doctor_experience`, `doctor_specialization_departments`, `doctor_specialization_symptoms`, `departments`, `symptoms`

Feature:

- Doctor profile with qualifications, experience, and working hours
- Departments & symptoms mapping → patient search doctor by specialty / symptom
- Doctor can update experience & specialization

Professional Realism:  High

- Allows complex search & filtering in real-life apps
-

5 Prescriptions & Medical History

Tables: `prescriptions`, `patients.medical_history`

Feature:

- Doctor issue prescription linked to appointment
- Patients medical history track
- Prescription download/print functionality

Professional Realism:  High

- Standard in telemedicine platforms
 - Ensure security & HIPAA compliance
-

6 Payments & Commissions

Tables: `payments`, `payment_methods`, `payment_status`, `commissions`

Feature:

- Payment record for appointments
- Admin commission tracking
- Payment status tracking
- Support multiple methods (card, wallet, etc.)

Professional Realism: ☒ High

- Can integrate Stripe / PayPal / Bkash
 - Required in professional apps
-

7 Reviews & Ratings

Tables: `doctor_reviews`

Feature:

- Patients doctor review & rating
- Average rating calculation
- Feedback tracking

Professional Realism: ☒ Medium-High

- Adds trust factor, common in real-life platforms
-

8 Chat / Messaging

Tables: `chat_messages`

Feature:

- Appointment-based messaging between patient & doctor
- Track sender, receiver, timestamp

Professional Realism:  Medium

- Real-time chat requires WebSocket / Django Channels
 - Database stores chat history
-

9 Audio / Video Calls

Tables: `call_sessions`

Feature:

- Track call metadata: start, end, duration, call type
- Appointment-based audio/video sessions

Professional Realism:  Medium

- Real streaming requires WebRTC / external service
 - Django/SQLite stores logs only, not live streaming
-

10 Audit / Logs

Tables: `audit_logs`

Feature:

- Track user actions (create, update, delete)

- Table name & record id logging
- JSON details for custom info

Professional Realism:  High

- Essential for accountability & debugging in production apps

11 Location & Hierarchy

Tables: **divisions**, **districts**, **upazilas**

Feature:

- Patient / doctor address management
- Filter doctor by district/division/upazila

Professional Realism:  High

- Realistic for Bangladesh-centric healthcare app
-

12 Overall Professional Capabilities

Feature	Real-life Professional Feasible?	Notes
User & Roles	 High	Admin, Doctor, Patient roles

Appointment Scheduling	✓ High	Conflict-free booking
Reminders	✓ Medium-High	Needs background task
Doctor Profile & Specializations	✓ High	Searchable, detailed
Prescriptions & Medical History	✓ High	Track & download
Payments & Commissions	✓ High	Gateway integration needed
Reviews & Ratings	✓ Medium-High	Optional but common
Chat	✓ Medium	Real-time needs WebSocket
Audio/Video Call	✓ Medium	Live streaming needs WebRTC
Audit Logs	✓ High	For accountability
Location Filter	✓ High	Bangladesh-specific hierarchy

✓ Conclusion:

- Database is very comprehensive → almost all professional features possible
- Some real-time features (chat, audio/video) require external services
- Using

MediSched – Telemedicine Management Software

1. Software Overview

MediSched হলো একটি ডিজিটাল হেলথ কেয়ার প্ল্যাটফর্ম, যা মূলত Patient-Doctor interaction সহজ ও নিরাপদ করার জন্য ডিজাইন করা হয়েছে। এই সফটওয়্যার দিয়ে appointment scheduling, online consultation, prescriptions, payments, reminders, reviews, এবং call sessions পরিচালনা করা যায়।

SQLite / Django দিয়ে এটি প্রোটোটাইপ আকারে তৈরি করা যাবে, এবং PostgreSQL ব্যবহার করলে প্রফেশনাল প্রডাকশনও সম্ভব।

2. Target Users

1. Patients

- স্থানীয় বা দূরবর্তী ডাক্তারের সাথে appointment নিতে পারেন
- Online chat, audio বা video call এর মাধ্যমে consultation নিতে পারেন
- Prescription এবং medical history track করতে পারবেন

2. Doctors

- Patient appointment এবং availability manage করতে পারবেন
- Experience, specialization, symptoms এবং qualification update করতে পারবেন
- Patient history ও prescriptions issue করতে পারবেন

3. Admins

- Platform এর management এবং user verification করবেন
 - Doctor verification, commissions tracking, audit logging করবে
 - Payment এবং transaction management দেখবেন
-

3. Key Features & Workflow

A. User Management

- Patients ও Doctors system এ register/login করবে
- Role-wise access control (Admin, Doctor, Patient)
- Doctors এর profile detailed (qualification, experience, specialization)
- Patients এর profile include personal info, blood group, address

B. Appointment Scheduling

- Patient Doctor select করবে এবং available slots অনুযায়ী appointment book করবে
- Doctor বা Patient cancel/update করতে পারবেন
- Appointment status tracked: pending, confirmed, completed

C. Reminders & Notifications

- Appointment এর আগে reminder পাঠানো হবে SMS, email বা push notification মাধ্যমে
- Reminder status trackable (sent, pending, failed)

D. Doctor Profile Management

- Doctor experience, specialization, symptoms add/update/delete করতে পারবেন
- Patients search/filter করতে পারবে doctor by specialty, location, symptoms

E. Prescriptions & Medical Records

- Doctor prescription issue করবে appointment অনুযায়ী
- Patients prescription download এবং medical history view করতে পারবেন
- Prescription history maintain

F. Payments & Commissions

- Patient payment করা যাবে online (Card, Wallet, Cash)
- Payment status: pending, completed, failed
- Admin commission automatically calculate

G. Reviews & Ratings

- Patient doctor কে rate ও review দিতে পারবেন
- Doctor profile এ average rating show হবে

H. Chat & Call Sessions

- Appointment-based messaging system
- Audio / Video call session log maintain (duration, type)
- Real-time communication optional via WebRTC integration

I. Audit Logs

- System-wide action tracking: create/update/delete
- Admin দেখতে পারবেন কোন user কোন activity করেছে
- Compliance এবং accountability ensure করবে

4. Business Impact & User Benefits

For Patients

- সহজভাবে Doctor search & booking
- Time-saving: hospital queue এ অপেক্ষা লাগবে না
- Teleconsultation: home থেকে medical advice

- Prescription & medical history organized & accessible

For Doctors

- Efficient appointment & schedule management
- Patient history access: diagnosis & treatment planning easier
- Increased reach to remote patients
- Track revenue & experience digitally

For Clinic / Hospital Admins

- Centralized management dashboard
- Monitor doctors, patients, payments, and commissions
- Audit trails maintain: accountability & reporting

For Business

- Scalable telemedicine platform
- Revenue from consultation fees & admin commissions
- Data-driven insights: popular symptoms, doctor demand, peak appointment times
- Online reputation via patient reviews & ratings

5. Revenue Model (Optional Business Plan)

1. Consultation Fee – Patient pays Doctor via platform
2. Admin Commission – Platform keeps % of fee
3. Subscription – Hospitals / Clinics pay subscription for system use

4. Ads / Promotions – Health product promotion within app
-

6. Professional-Level Software Outcome

- Realistic Bangladesh-specific telemedicine app
- Patient, Doctor, Admin management centralized
- Appointment + Consultation (Chat / Audio / Video)
- Prescriptions + Payments + Reviews + Reminders
- Audit trail ensures security and compliance
- Scalable from prototype (SQLite) to production (PostgreSQL + Celery + WebRTC)

✓ Conclusion:

MediSched এর মাধ্যমে patients doctors এর সাথে সহজে connect করতে পারবেন, doctors efficiently manage করবেন patients & appointments, এবং admins পুরো system control করবেন। এটি প্রফেশনাল telemedicine software হিসেবে বাস্তবায়নযোগ্য।

1 General Guideline

Django-তে বড় project একাধিক apps-এ ভাগ করলে maintain করা সহজ হয়।
তোমার project-এর modules অনুযায়ী:

1. users – User management (patients, doctors, admins, roles)
2. appointments – Appointment scheduling, status, reminders
3. doctors – Doctor profile, experience, specializations, symptoms
4. patients – Patient profile, medical history, prescriptions
5. payments – Payment methods, transactions, commissions

6. reviews – Doctor reviews & ratings
7. chat – Messaging between patient & doctor
8. calls – Audio / Video call sessions
9. audit – Audit logs, system actions

Optionally, তুমি core / lookup app বানাতে পারো যেখানে divisions, districts, upazilas, genders, roles, departments, symptoms ইত্যাদি থাকবে।

2 Suggested Apps & Responsibilities

App Name	Purpose / Features
users	User auth (login/logout/register), roles, permissions, profile management, JWT/session auth
patients	Patient-specific info: medical history, prescriptions, address, blood group
doctors	Doctor-specific info: profile, qualification, experience, specialization, working hours, symptoms
appointments	Book/cancel/update appointments, track status, reminders (Celery/Redis optional)
payments	Payment records, payment methods, commissions, integration with Stripe/Bkash
reviews	Add, list, update doctor reviews and ratings
chat	Appointment-based messaging, chat history (real-time optional via Django Channels)
calls	Call session logs (audio/video) via WebRTC/Jitsi integration
audit	Track user actions: create/update/delete, table & record ID, JSON details
core / lookup	Divisions, districts, upazilas, departments, symptoms, genders, roles

3 Advantages of this Structure

- Modular → maintenance সহজ
- Permissions and roles controlled centrally in users app
- App-specific models, views, templates, URLs organized cleanly
- Future scalability → যেমন PostgreSQL + Celery + WebRTC integration সহজ

```

medisched/          # Project root
├── manage.py        # Django manage script
├── env/              # Virtual environment (optional)
├── medisched/        # Project settings module
│   ├── __init__.py
│   ├── asgi.py
│   ├── settings.py   # Main settings
│   ├── urls.py        # Root URLconf
│   └── wsgi.py
├── home/             # Home app (landing page, static pages)
│   ├── migrations/
│   ├── templates/
│   │   └── home/
│   │       └── index.html
│   ├── static/
│   │   ├── home/
│   │   │   ├── css/
│   │   │   │   └── style.css
│   │   │   └── js/
│   │       └── main.js
│   ├── __init__.py
│   ├── admin.py
│   ├── apps.py
│   ├── models.py
│   ├── tests.py
│   ├── urls.py
│   └── views.py
├── users/            # Users app (auth, dashboards)
│   ├── migrations/
│   └── templates/

```

```
|
|
|   └─ users/
|       ├── doctor_dashboard.html
|       ├── patient_dashboard.html
|       └── admin_dashboard.html
|
|   └─ static/
|       └─ users/
|           ├── css/
|           └── js/
|
|   ├── __init__.py
|   ├── admin.py
|   ├── apps.py
|   ├── models.py           # CustomUser model with roles
|   ├── tests.py
|   ├── urls.py             # Users app URLs
|   └── views.py
|
|   ├── templates/         # Global templates (if any)
|   ├── static/            # Global static files (if any)
|   └── requirements.txt    # pip dependencies
```